

	UNIVERSIDAD DON BOSCO FACULTA DE INGENIERIA ESCUELA DE COMPUTACION
Ciclo I	Desarrollo de aplicaciones con Web Frameworks Spring Security

I. OBJETIVOS.

- Que el estudiante sea capaz de analizar, diseñar y construir microservicios, aplicando todos los conocimientos adquiridos.
- Que estudiante sea capaz de resolver problemas utilizando programación con Spring Boots.

II. INTRODUCCION

Spring Security es un framework que proporciona autenticación, autorización y protección contra ataques comunes. Con un soporte de primera clase para proteger aplicaciones imperativas y reactivas, es el estándar de facto para proteger aplicaciones basadas en Spring.

Spring Security requiere un entorno de ejecución Java 17 o superior.

Dado que Spring Security funciona de forma autónoma, no es necesario colocar ningún archivo de configuración especial en el entorno de ejecución de Java. En particular, no es necesario configurar un archivo de política especial del Servicio de Autenticación y Autorización de Java (JAAS) ni colocar Spring Security en rutas de clases comunes.

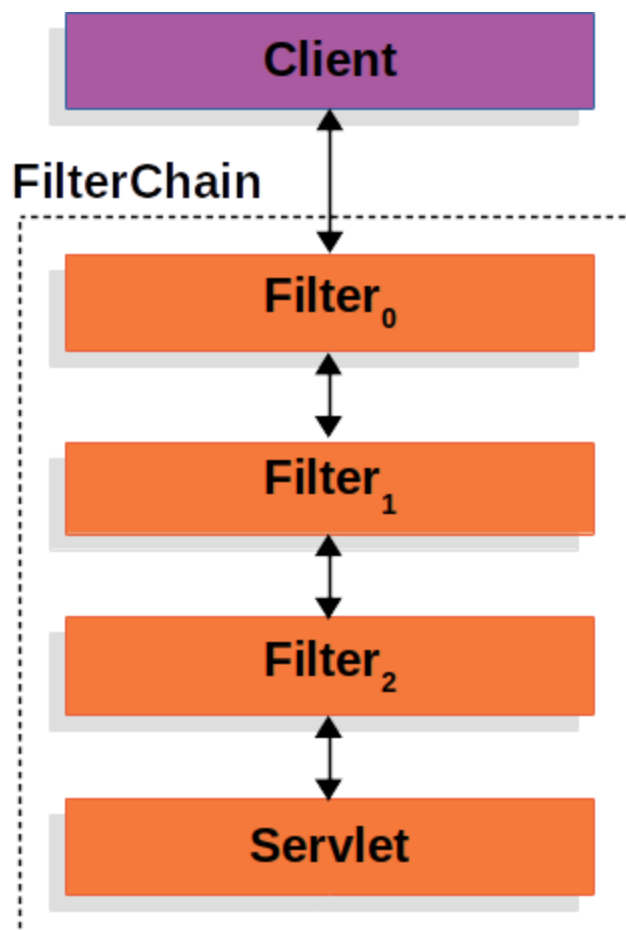
Spring Boot proporciona un spring-boot-starter-security iniciador que agrega las dependencias relacionadas con Spring Security

```
<dependencies>
  <!-- ... other dependency elements ... -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
</dependencies>
```

Arquitectura

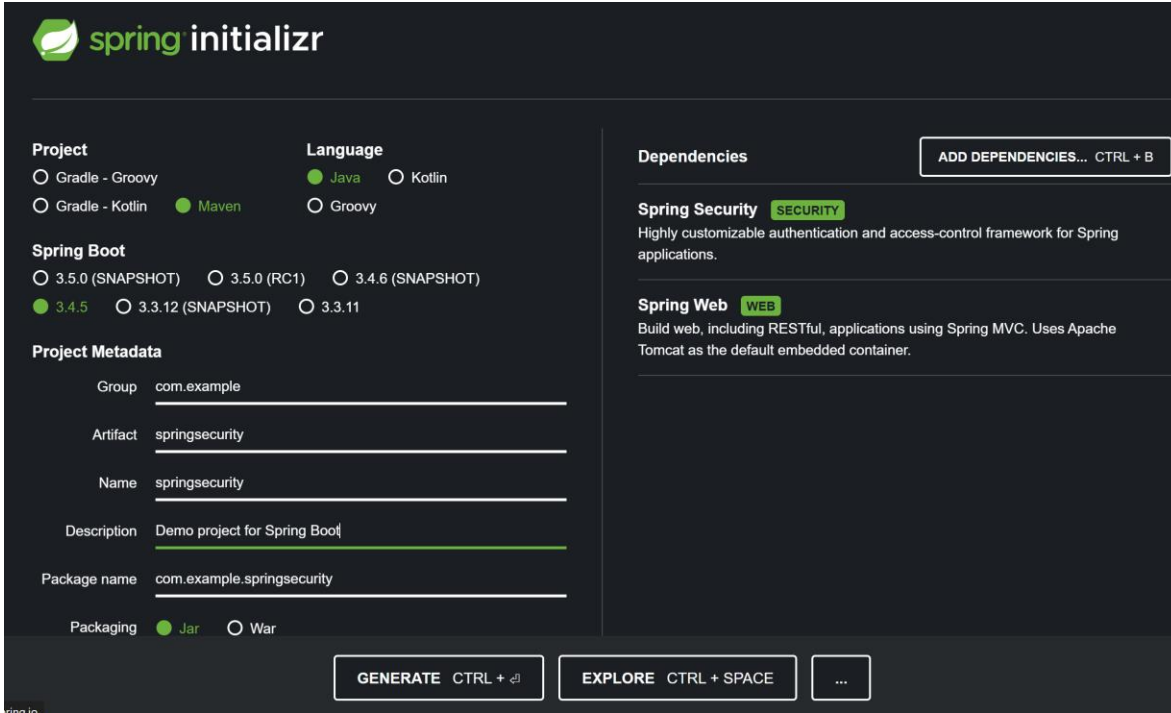
La compatibilidad de Spring Security con Servlets se basa en Filtros de Servlets, por lo que conviene analizar primero la función general de los Filtros. La siguiente imagen muestra la estructura típica de los controladores para una sola solicitud HTTP.

El cliente envía una solicitud a la aplicación y el contenedor crea un [nombre de la instancia FilterChain] que contiene las Filterinstancias y Servletque debe procesar el [nombre HttpServletRequestde la instancia], según la ruta de la URI de la solicitud. En una aplicación Spring MVC, [Servletnombre de la instancia] es una instancia de [nombre de la instancia] DispatcherServlet. Como máximo, se Servletpuede manejar un [nombre de la instancia] HttpServletRequesty HttpServletResponse[nombre de la instancia]. Sin embargo, Filterse pueden usar más de uno para:



III. PROCEDIMIENTO

PASO 1.



The Spring Initializr web form is shown with a dark theme. It includes sections for Project, Language, Spring Boot, Project Metadata, Dependencies, and buttons for GENERATE, EXPLORE, and ADD DEPENDENCIES.

Project

☐ Gradle - Groovy ☒ Gradle - Kotlin ☒ Maven ☐ Groovy

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 3.5.0 (SNAPSHOT) ☐ 3.5.0 (RC1) ☐ 3.4.6 (SNAPSHOT) ☒ 3.4.5 ☐ 3.3.12 (SNAPSHOT) ☐ 3.3.11

Project Metadata

Group:

Artifact:

Name:

Description:

Package name:

Packaging: ☒ Jar ☐ War

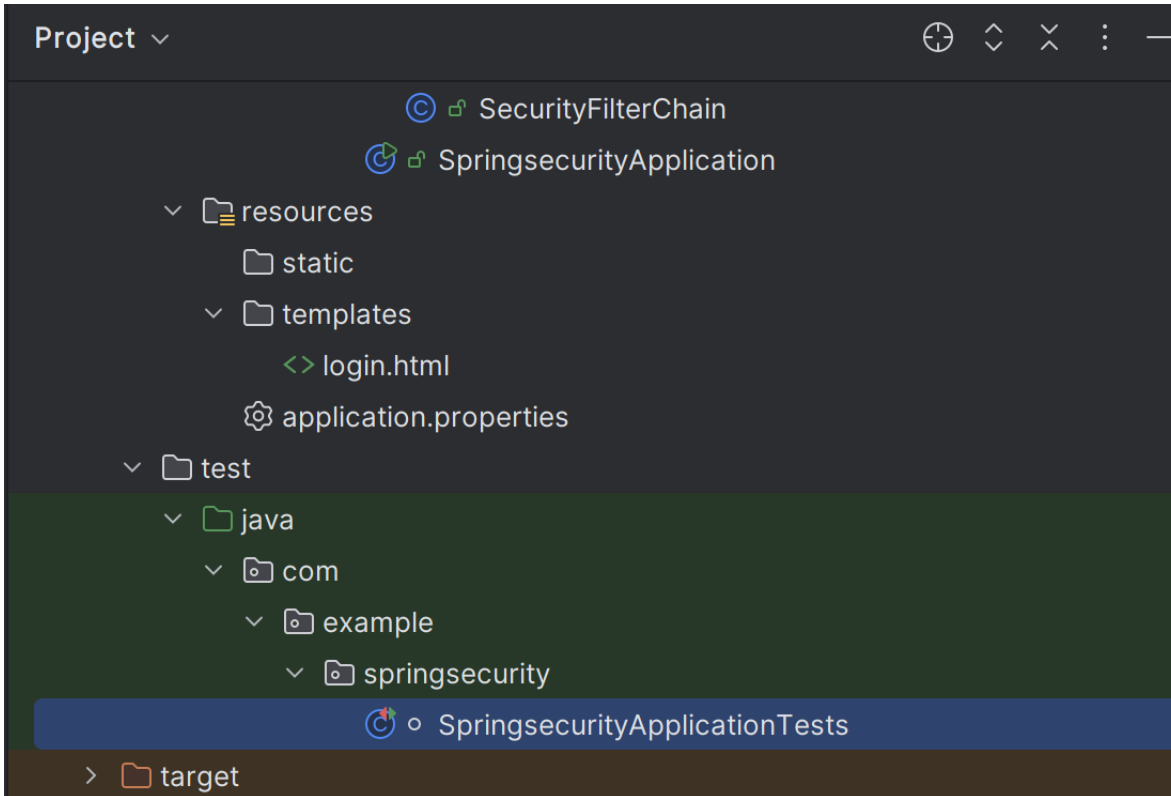
Dependencies

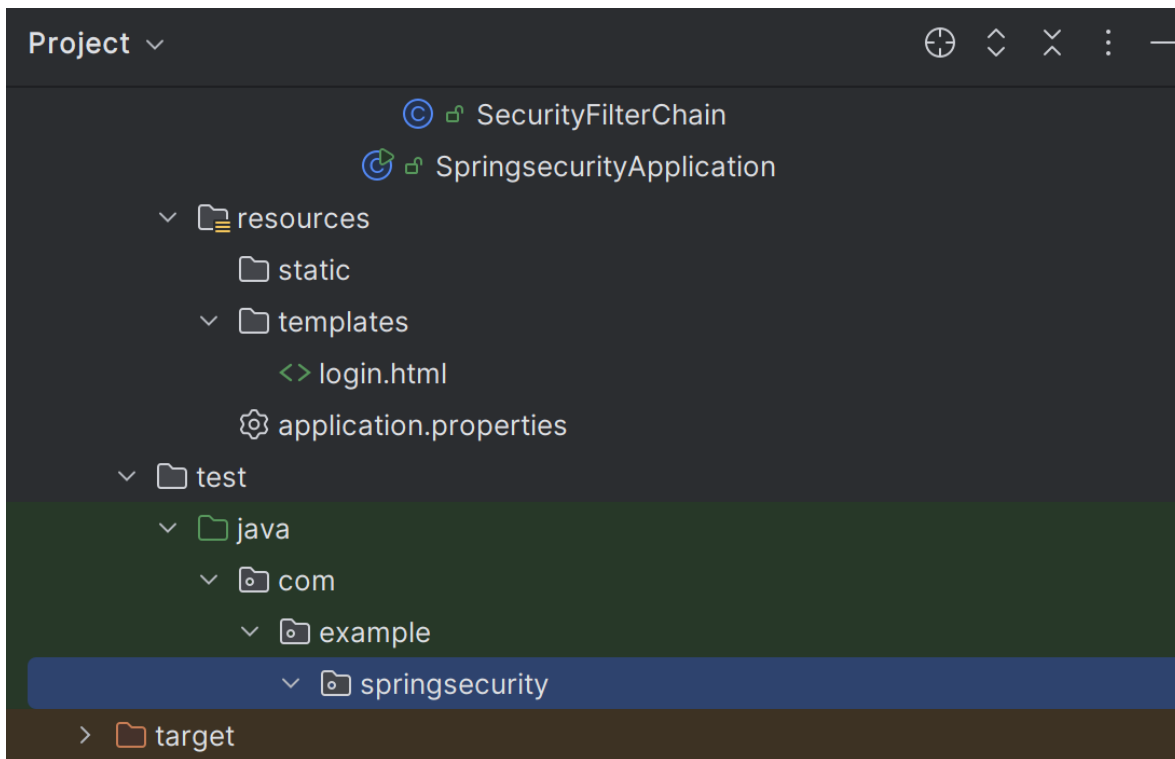
Spring Security SECURITY
Highly customizable authentication and access-control framework for Spring applications.

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

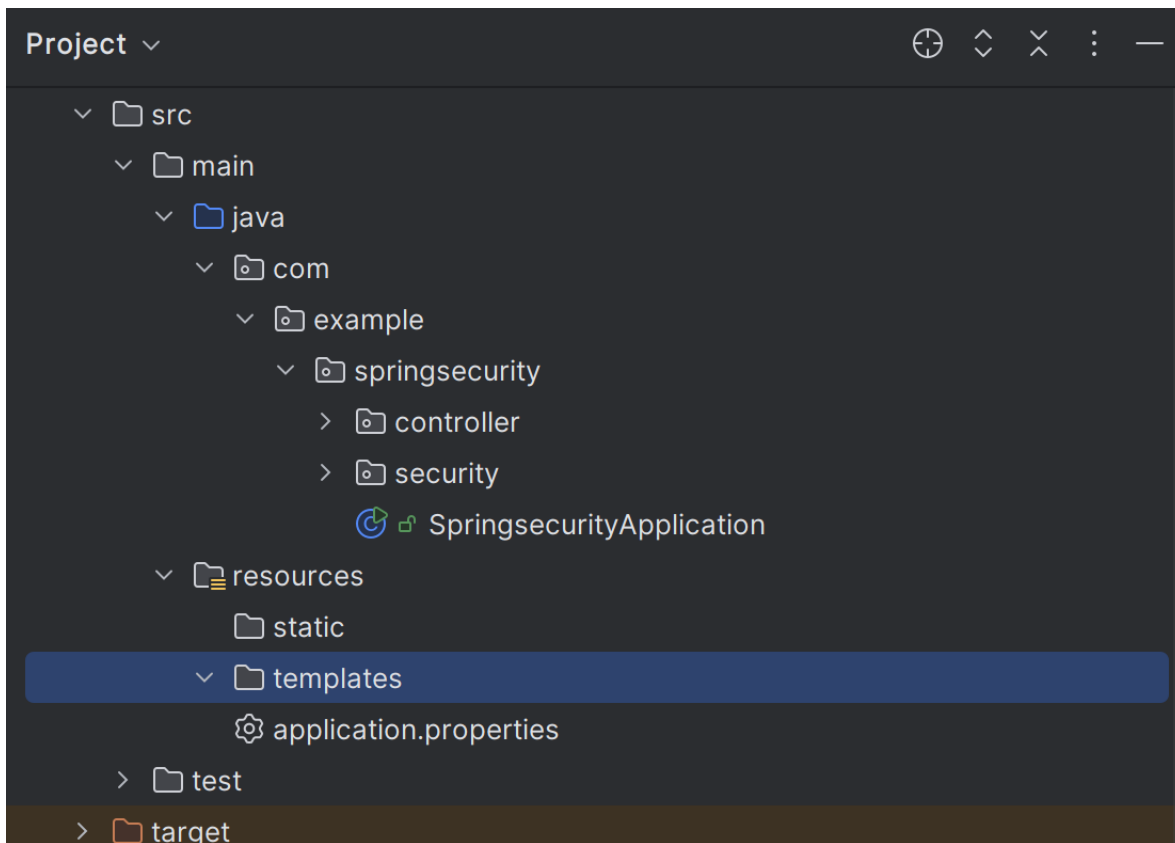
Buttons: GENERATE (CTRL + G), EXPLORE (CTRL + SPACE), ADD DEPENDENCIES... (CTRL + B)

PASO 2: Eliminamos la clase Test

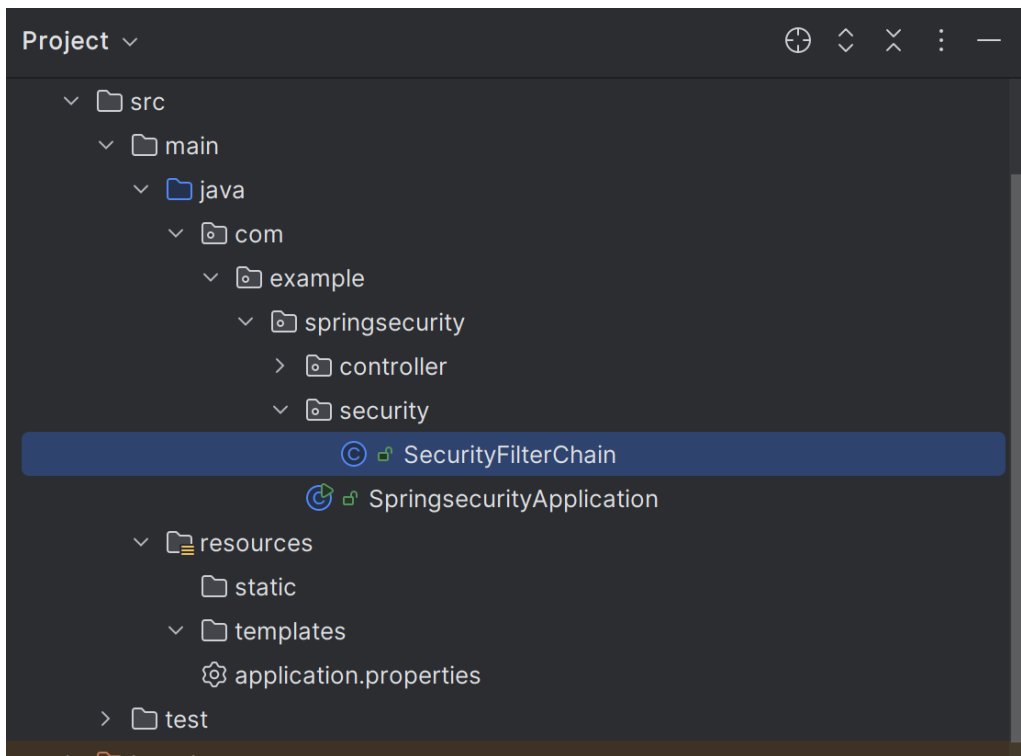




PASO 3: ESTRUCTURA DE PROYECTO



PASO 4: CREAMOS LA CLASE “SecurityFilterChain” en el paquete “security”



ESTRUCTURA DE LA CLASE “SecurityFilterChain”

```
package com.example.springsecurity.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.DefaultSecurityFilterChain;
import static org.springframework.security.config.Customizer.withDefaults;

@Configuration
@EnableWebSecurity
public class SecurityFilterChain {

    @Bean
    public DefaultSecurityFilterChain filterChain(HttpSecurity http) throws Exception {...}

    @Bean
    //public UserDetailsService Users(){
    public InMemoryUserDetailsManager Users(){...}

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Spring Bean : “DefaultSecurityFilterChain”

```
@Bean no usages
public DefaultSecurityFilterChain filterChain(HttpSecurity http) throws Exception{
    http
        .csrf(Customizer.withDefaults())
        .authorizeHttpRequests( AuthorizationManagerRequestMat... authorize -> authorize
            .requestMatchers( ...patterns: "/private").hasRole("USER")
            .requestMatchers( ...patterns: "/private/").denyAll()
            .requestMatchers( ...patterns: "/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .logout(Customizer.withDefaults())
        .exceptionHandling(withDefaults())
        .formLogin(withDefaults())
        .httpBasic(withDefaults());
    return http.build();
}
```

Spring Bean: “InMemoryUserDetailsManager” Este método es utilizado para simular los usuarios del sistema.

```
@Bean no usages
//public UserDetailsService Users(){
public InMemoryUserDetailsManager Users(){
    User.UserBuilder users = User.builder();

    var publico = User.withUsername("publico")
        .password(passwordEncoder().encode( rawPassword: "password"))
        //.roles("ADMIN")
        .build();

    var privado = User.withUsername("privado")
        .password(passwordEncoder().encode( rawPassword: "password"))
        .roles("USER")
        .build();

    var admin = User.withUsername("admin")
        .password(passwordEncoder().encode( rawPassword: "password"))
        .roles("ADMIN")
        .build();

    return new InMemoryUserDetailsManager(publico,privado,admin);
}
```

Spring Bean: “PasswordEncoder” para encriptar las claves

```
@Bean 3 usages
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

PASO 5: CREAMOS LA CLASE “HelloController” en el paquete “controller”

```
package com.example.springsecurity.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController  no usages
public class HelloController {

    @GetMapping("/")  no usages
    public String index() {
        return "Esta es la pagina de Inicio";
    }

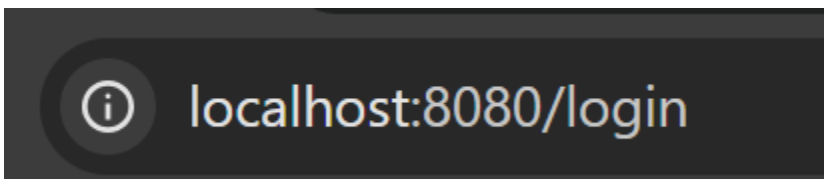
    @GetMapping("/public")  no usages
    public String publicEndPoint() { return "Este endpoint es Publico"; }

    @GetMapping("/private")  no usages
    public String privateEndPoint() { return "Este endpoint es Privado"; }

    @GetMapping("/admin")  no usages
    public String adminEndPoint() { return "Este endpoint es Administrador"; }

}
```

PASO 6: Ejecutar la aplicación.



En el navegador veremos el formulario por defecto del framework de Spring.

Please sign in

Sign in

Ejecutar pruebas con los diferentes usuarios.

IV. Mini Desafío

Tomando como base la guía 9, incorpora la seguridad para que:

1. El usuario “sololectura” pueda únicamente acceder a los microservicios GET de lectura por ID y lectura de todos los registros
2. El usuario “soloescritura” pueda únicamente acceder a los servicios POST, PUT, DELETE, para crear, modificar y leer.
3. El usuario “admin” tenga todos lo permisos.